

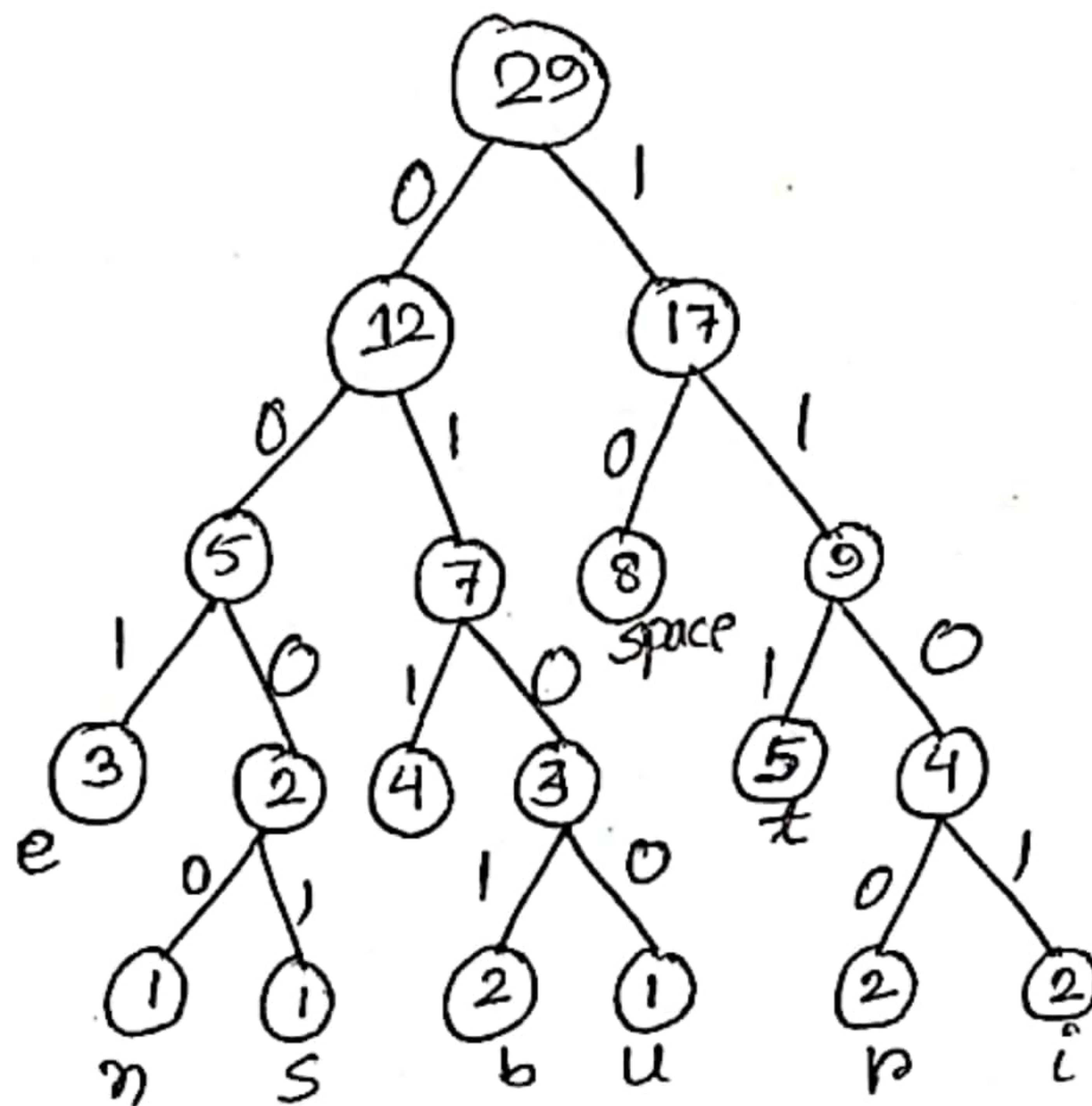
Spring-18 Ques. SolveAnswer to the ques. no 1:-(a) no.

Difference between dynamic programming and greedy algorithm:-

Dynamic Programming	Greedy Algorithm
1) Dynamic Programming solves subproblems recursively and reuses their solutions to avoid repeated calculations.	1) The greedy algorithm makes locally optimal choices at each step without considering the future consequences.
2) It is slower and more complex than greedy approach.	2) It is faster and simple.
3) In Dynamic Programming, we choose at each step, but the choice may depend on the solution to sub-problems.	3) In a greedy algorithm, we make whatever choice seems best at the moment and then solve the subproblems arising after the choice is made.
4) Less efficient as compared to greedy approach.	4) More efficient.
5) Follows bottom up and top down approach.	5) Follows top down approach.

1(b) no:

The Huffman tree is:



The Huffman code for next "to be or not to be it is true" is given below,

111 0111 001 01001100 1111 00000000 1111 110111 0010
10011 01101111101 101 00011011 1111000 100001

1 (c) no:

Activity Selection Problem: —

Activity Selection Problem is a combinatorial optimization problem concerning the selection of non-conflicting activities to perform within a given time frame, given a set of activities each marked by start time (s_i) and finish time (f_i).

Step 1:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

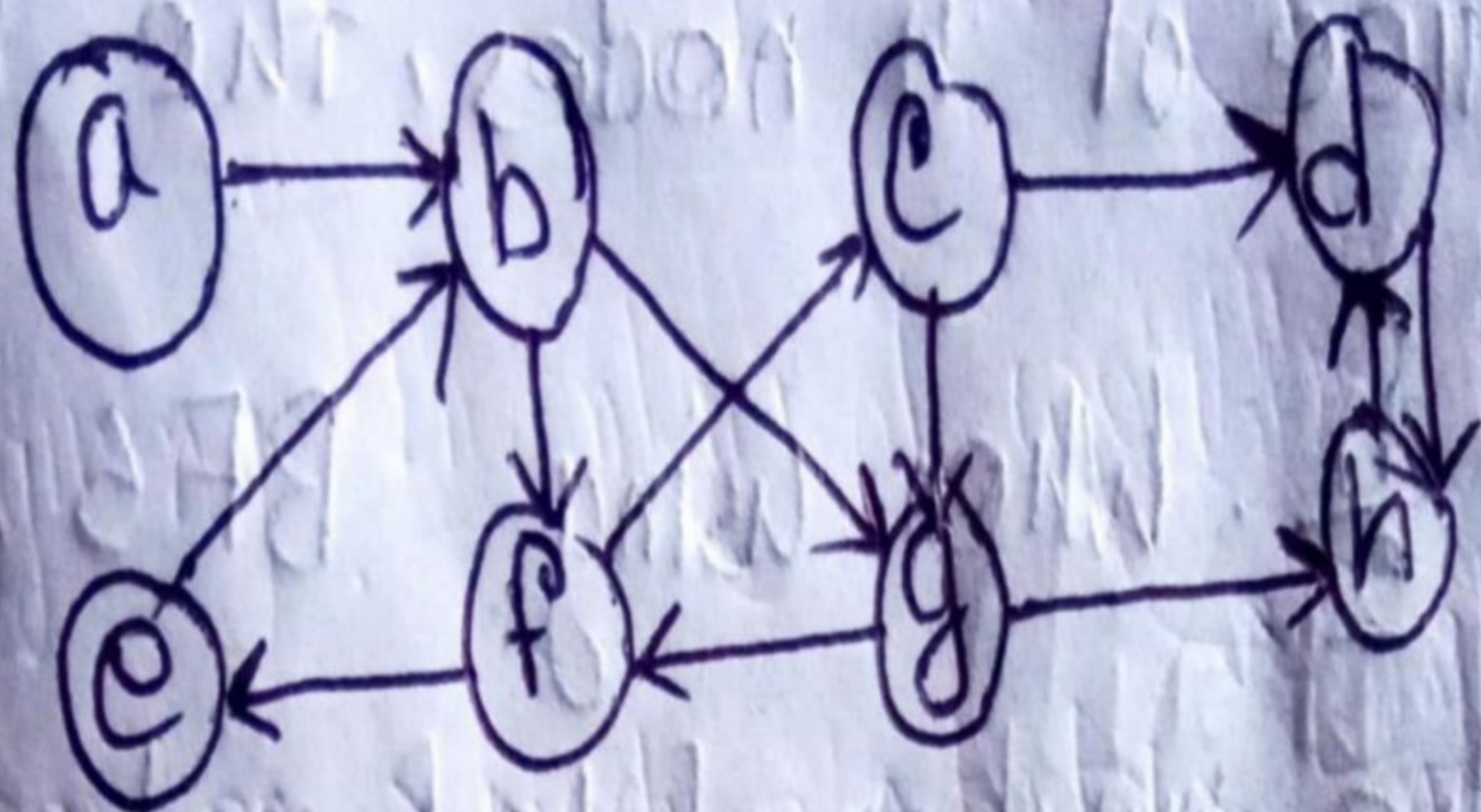
Step 1: Sorting f_i in ascending order.

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

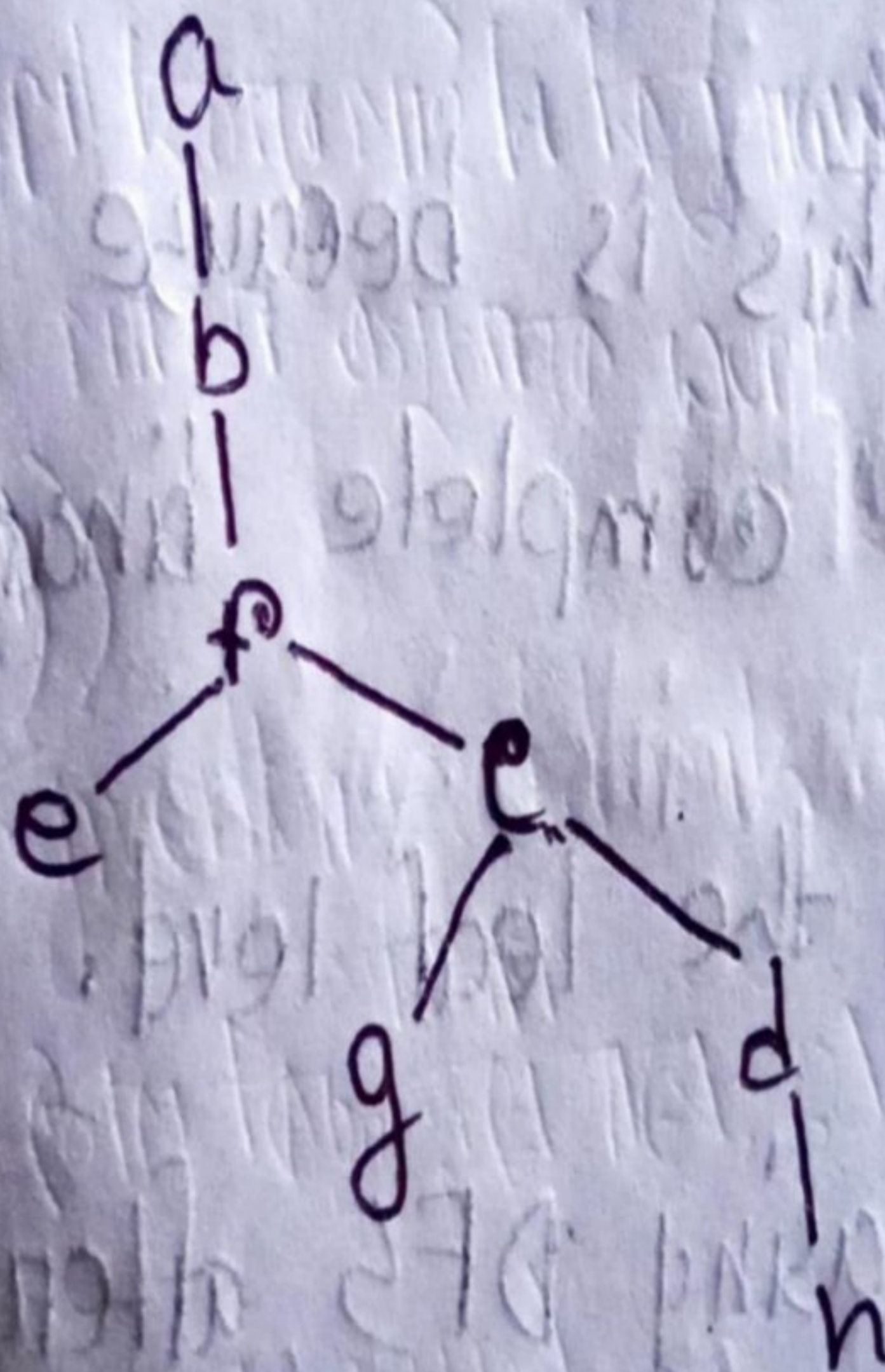
Step 3:

i	1	4	8	11
s_i	1	5	8	12
f_i	4	7	11	14

Ans to the Q. 2(c)



Ans:



Ans to the Q. No 2(b)

In a complete binary tree of n nodes, the number of leaf nodes is approximately $\frac{n}{2}$. When using BFS, after visiting half of the vertices, the number of leaf nodes visited will also be approximately $\frac{n}{2}$. This is because BFS visits nodes level by level and in a complete binary tree, half of the nodes are located in the leaf level.

On the other hand, when using DFS, after visiting half of the vertices, the number of leaf nodes visited may vary. DFS explores a path as deeply as possible before backtracking, so it may not visit nodes in a level-wise manner. Depending on the structure of the complete binary tree and the order in which the nodes are visited, DFS may or may not visit half of the leaf nodes after visiting half of the vertices.

Ans to the Q. No 2(c)

i) void disconnectVertex(int v)

{ int i, j;

for (i=1; i<=v; i++) {

graph[i][3] = 0;

graph[3][i] = 0;

}}

ii) int IN-DEG[v+1];

int Out-DEG[v+1];

vector<int> moreInward;

void computeDeg(int v)

{ int i, j;

for (i=1; i<=N; i++)

{ for (j=1; j<=v; j++)

{ if (graph[j][i] == 1)

IN-DEG++;

}

else if (graph[i][j] == 1)

Out-DEG++;

}

}

}


```
void createList(int v)
```

```
{ int i;
```

```
for (i=1; i<=v; i++)
```

```
{ if (IN_DEG[i] > OUT_DEG[i])
```

```
moreInward.push_back(i);
```

```
else if (moreInward.empty())
```

```
printf("There is no such vertex.\n");
```

```
else
```

```
{ for (i=0; i<moreInward.size(); i++)
```

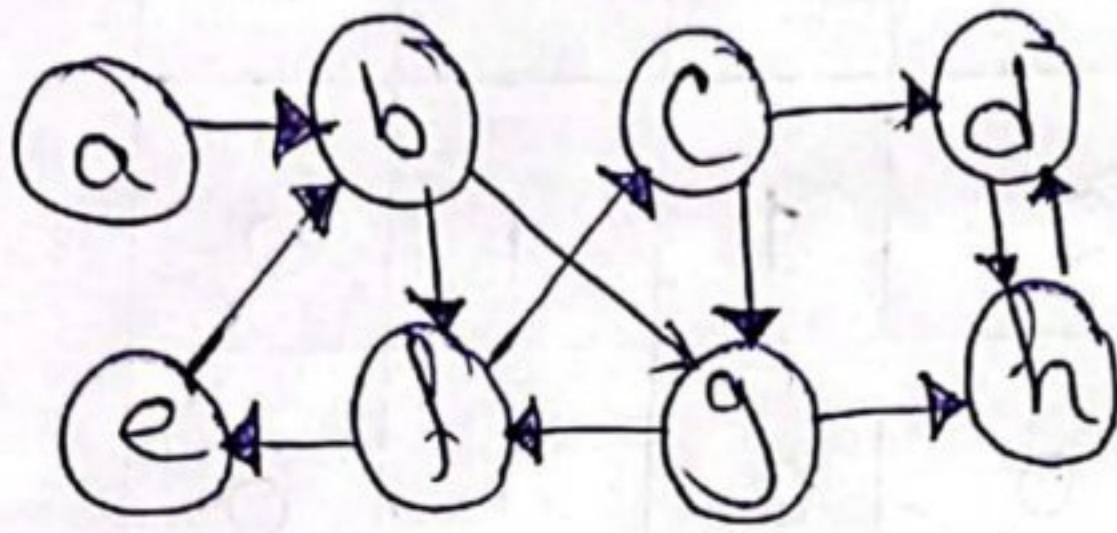
```
printf("%.d\t", moreInward[i]);
```

```
}
```

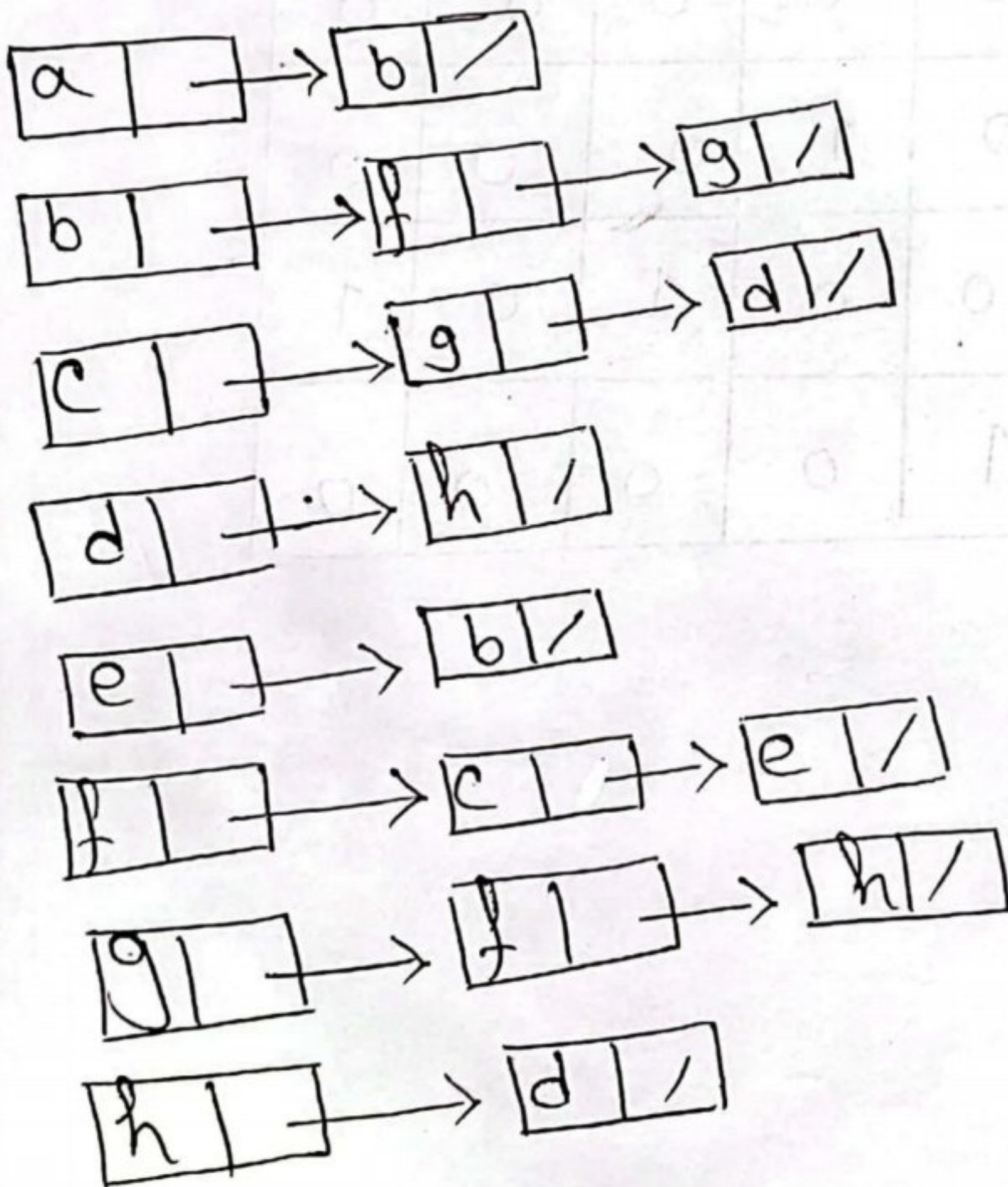
```
}
```


3(A)

show the adjacency list and adjacency matrix of the graph



The adjacency list :-

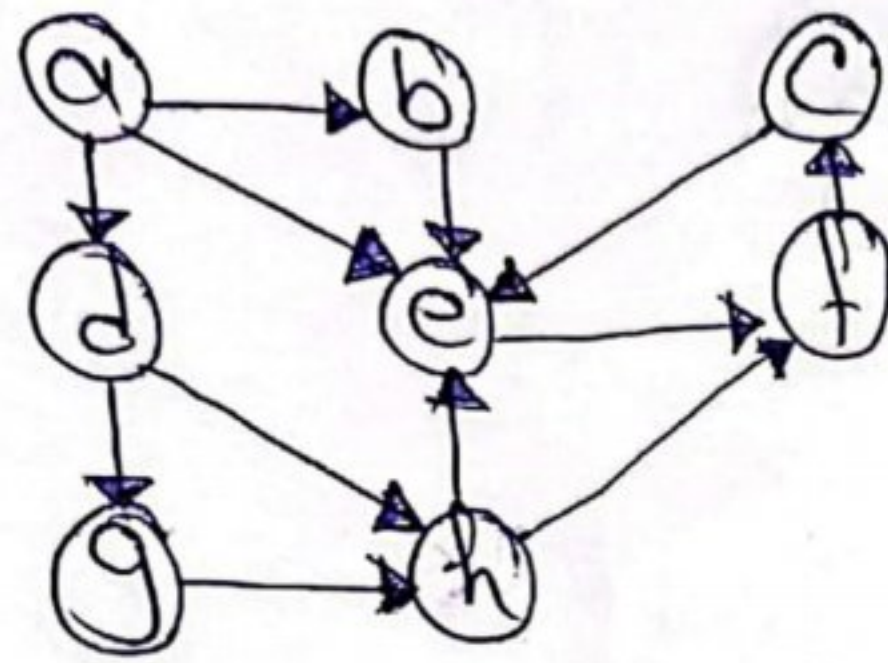


The adjacency matrix

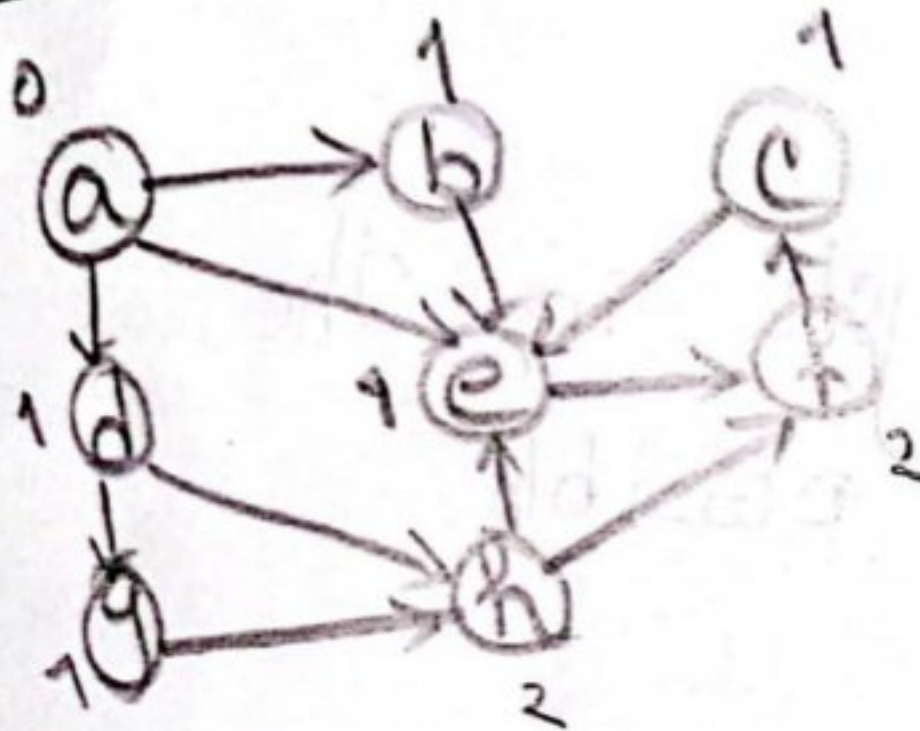
	a	b	c	d	e	f	g	h
a	0	1	0	0	0	0	0	0
b	0	0	0	0	0	1	1	0
c	0	0	0	1	0	0	1	0
d	0	0	0	0	0	0	0	1
e	0	1	0	0	0	0	0	0
f	0	0	1	0	1	0	0	0
g	0	0	0	0	0	1	0	1
h	0	0	0	1	0	0	0	0

3(c)

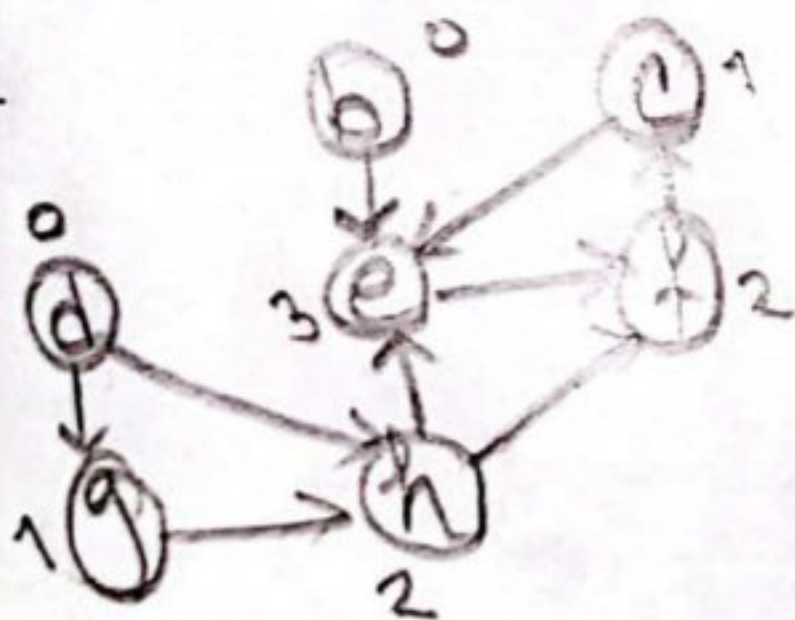
Show topological sort of the following graph



Step-1

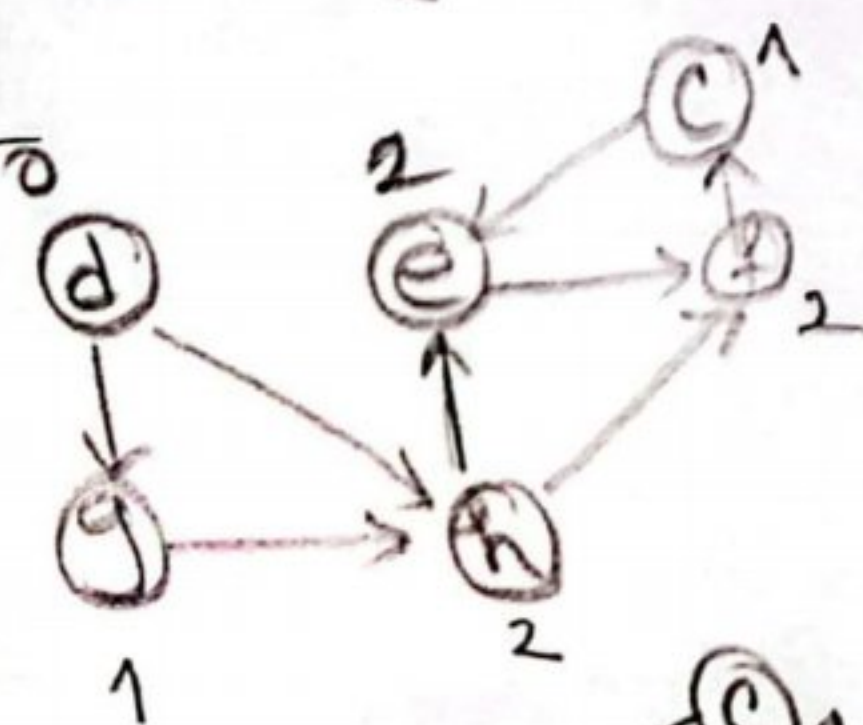


Step-2



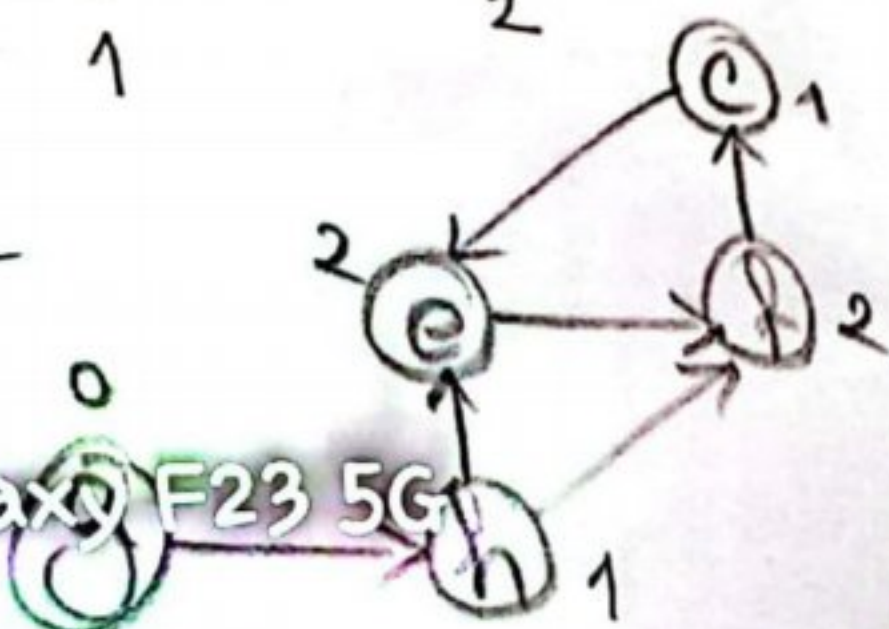
a b

Step-3



a b c

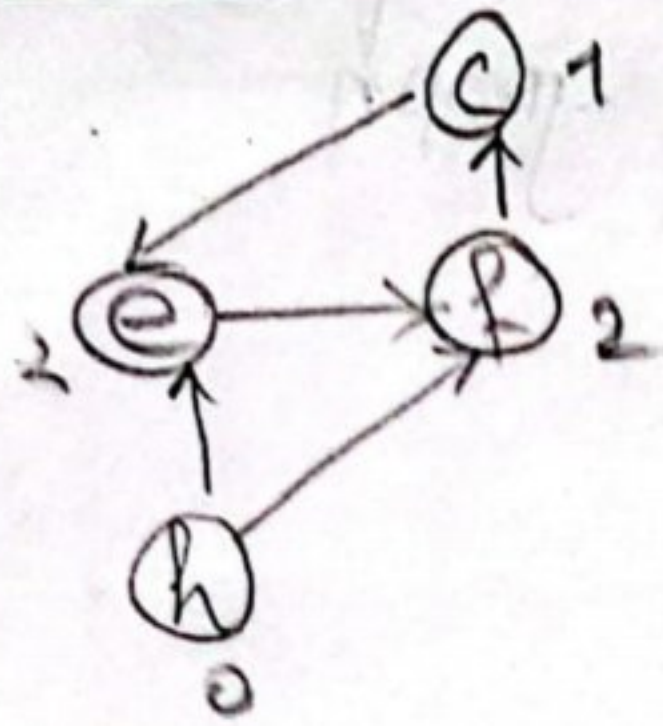
Step-4



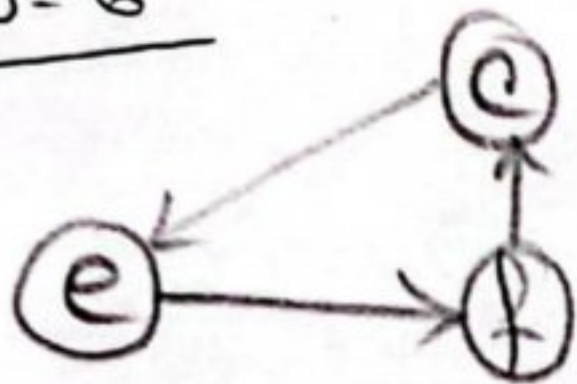
a b c d

Galaxy F23 5G

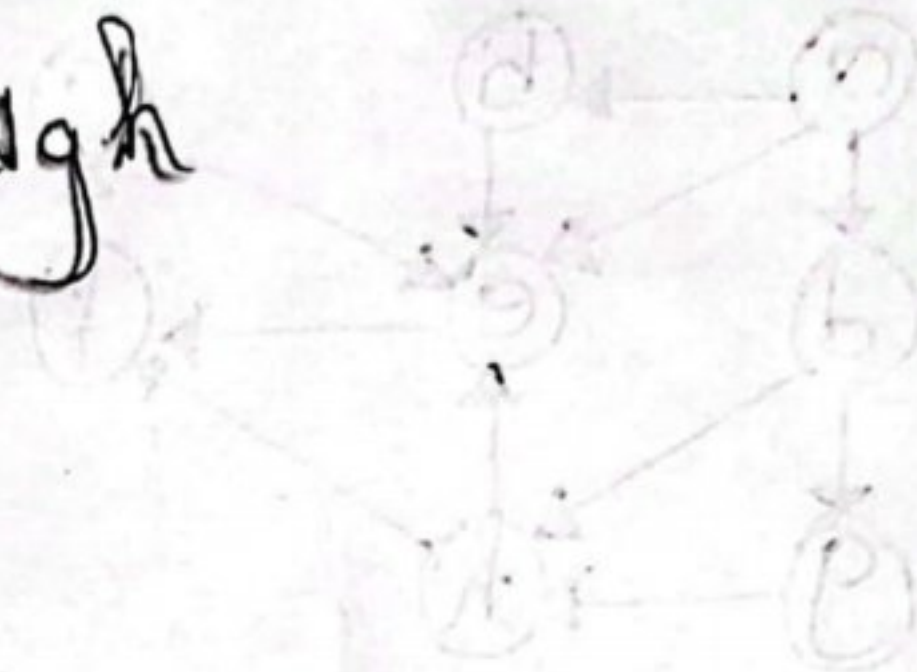
Step-5



Step-6



abdg h



abdg h - - -

As we can see there is a cycle in the graph so topological sort is not possible

Ans. to Ques. No. 4

(a)

Proof:- Consider a graph with edges and vertices.

Let P_{jk} be the shortest path from vertex v_i to vertex v_k .

$\therefore P_{jk} = \langle v_i, \dots, v_k \rangle$ be a shortest path from v_i to v_k .

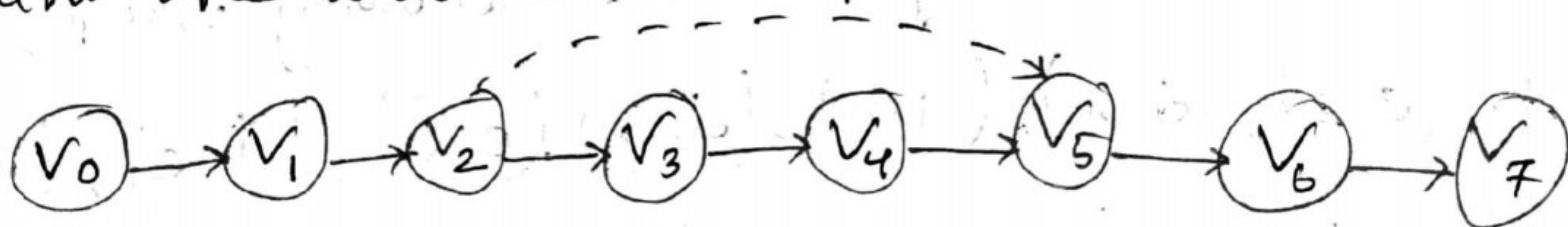
Now consider P_{ij} as a subpath of P_{jk} ,

$\therefore P_{ij} = \langle v_i, \dots, v_j \rangle$ be a subpath of P_{jk} from v_i to v_j for any i, j .

Since the weight of the P_{jk} path is already minimized, the weight of P_{ij} path will be minimized as well.

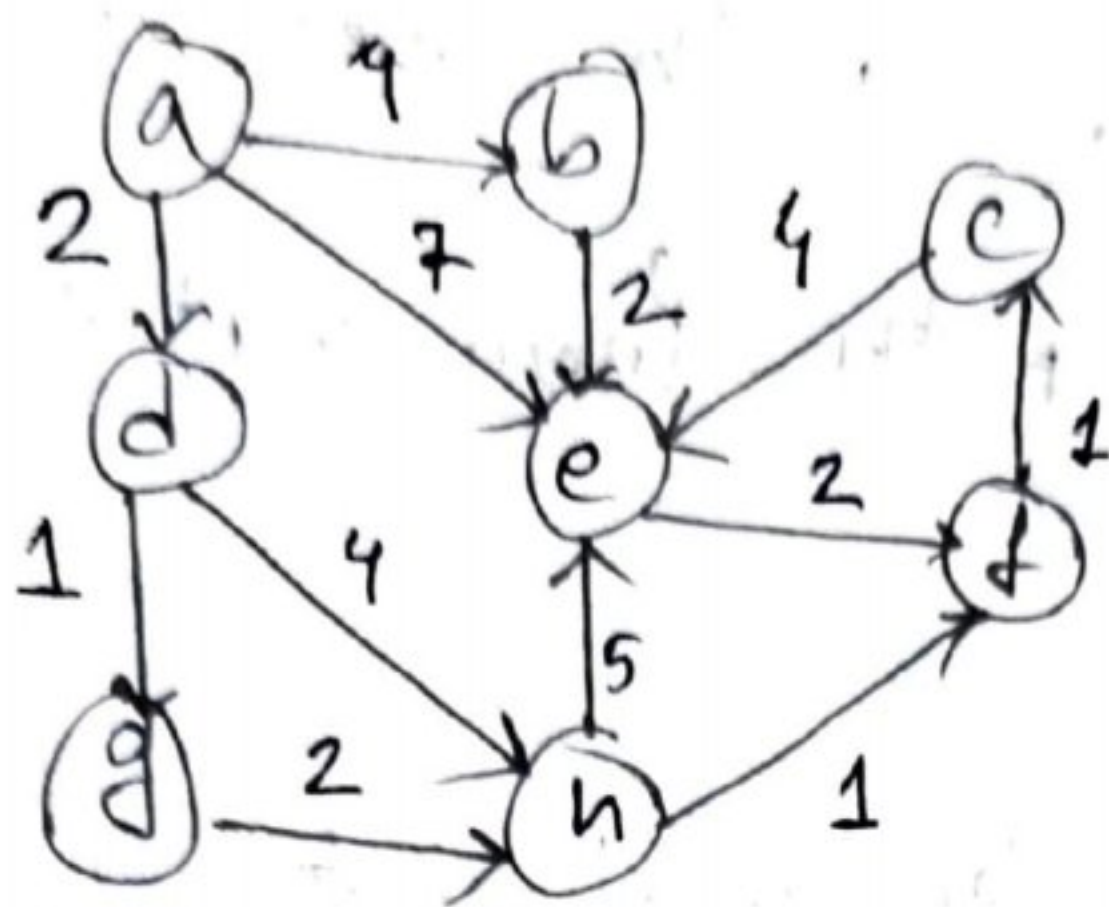
Then, P_{ij} is a shortest path from v_i to v_j .

And it is also proved that subpaths of shortest paths are also shortest paths.

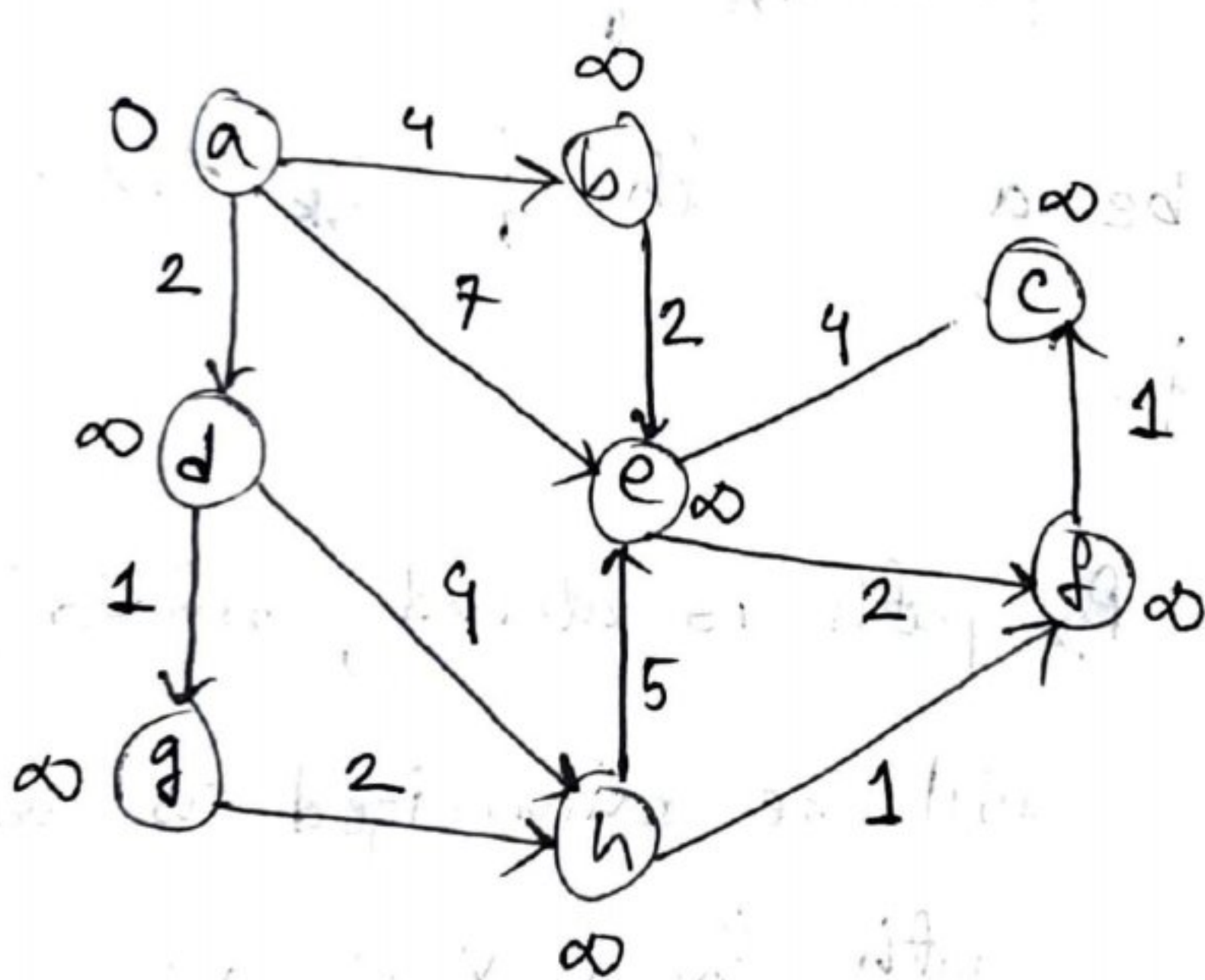


(b)

Given,



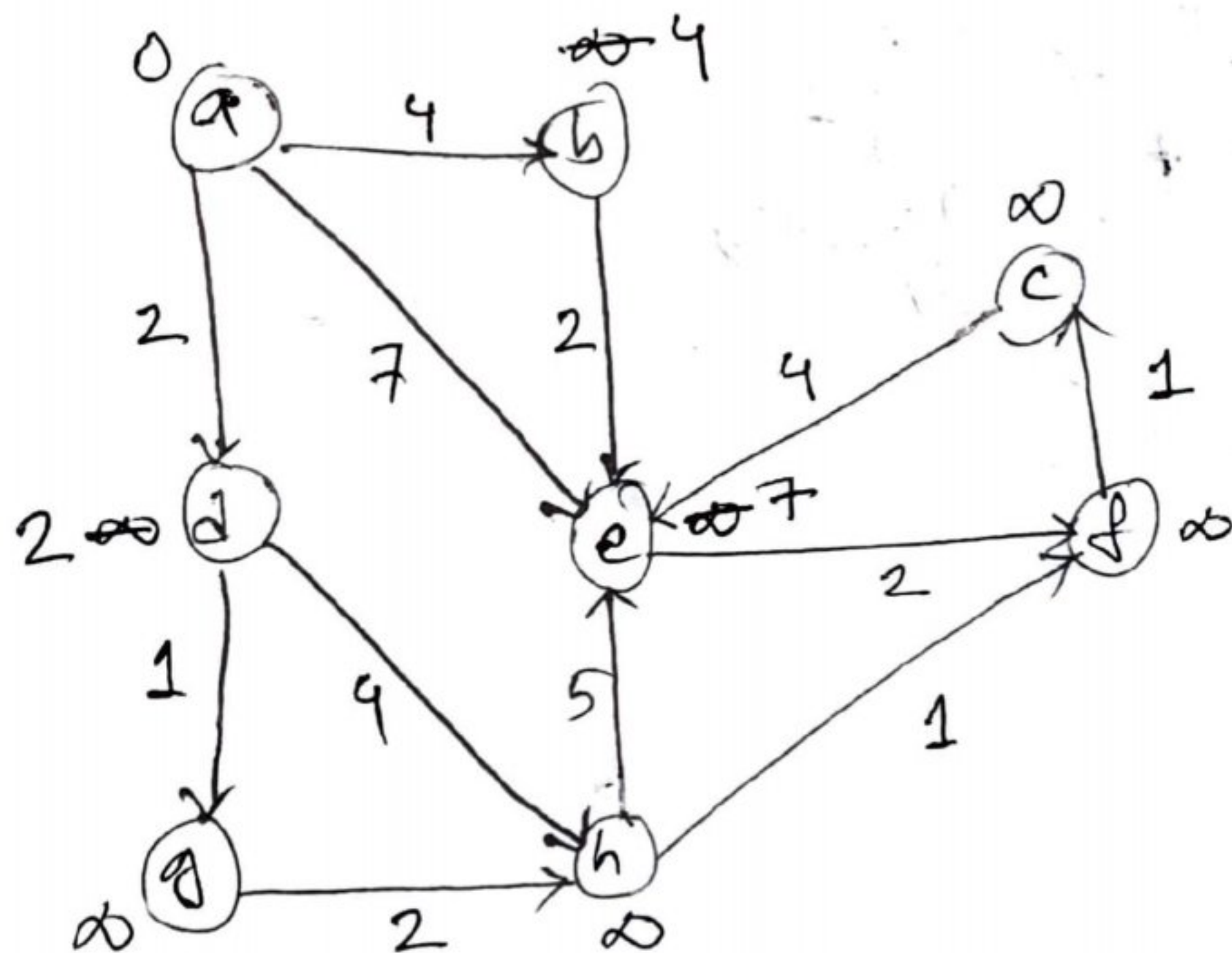
Let, source vertex $a=0$ and other vertices as ∞



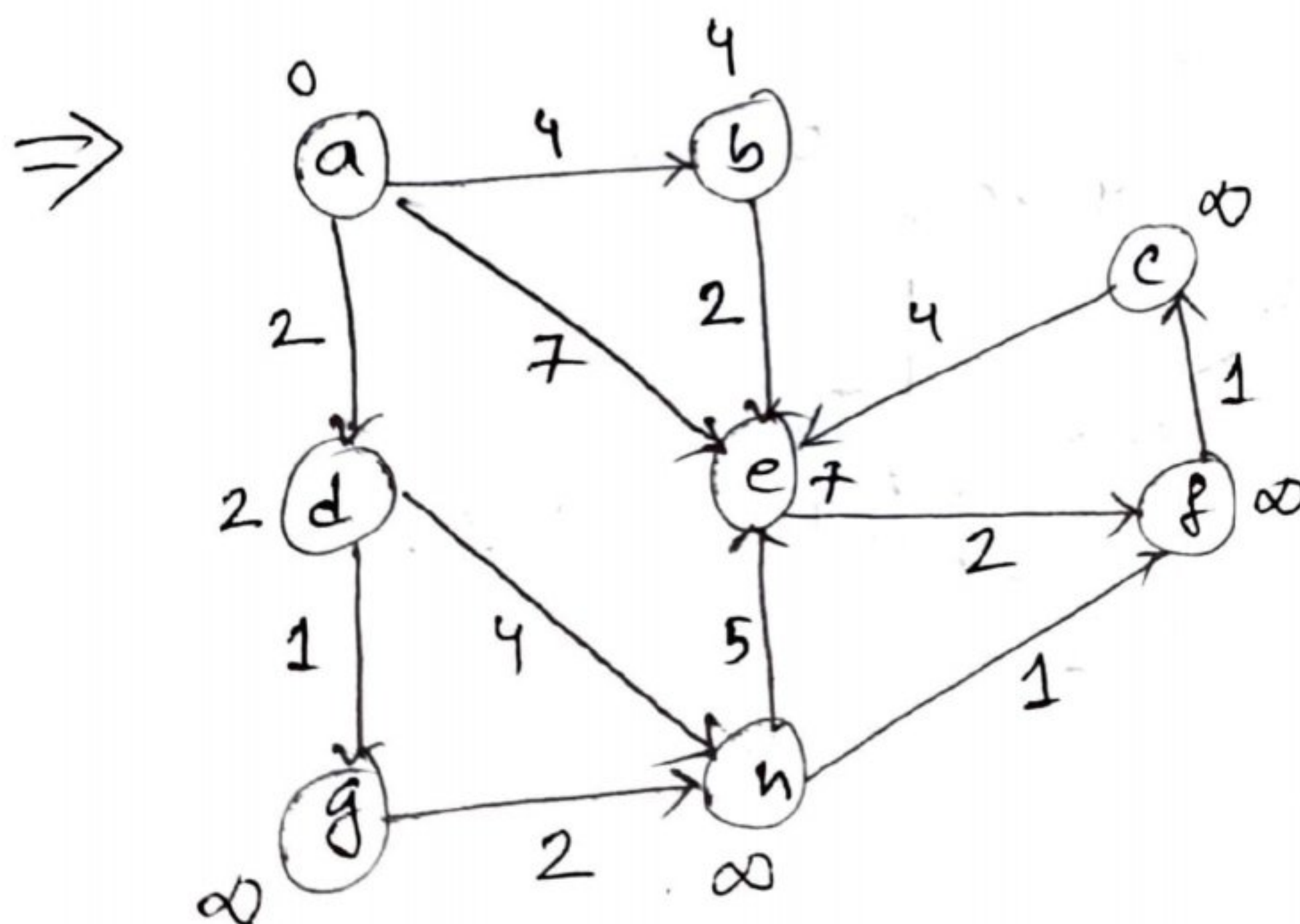
now, applying Bellman-Ford's shortest path algorithm by relaxing the edges in the following order,
(h,e), (c,e), (d,h), (g,h), (d,g), (e,f), (f,c), (h,f),
(g,e), (b,e), (a,b), (a,d)

and showing only first two iterations:-

(1)

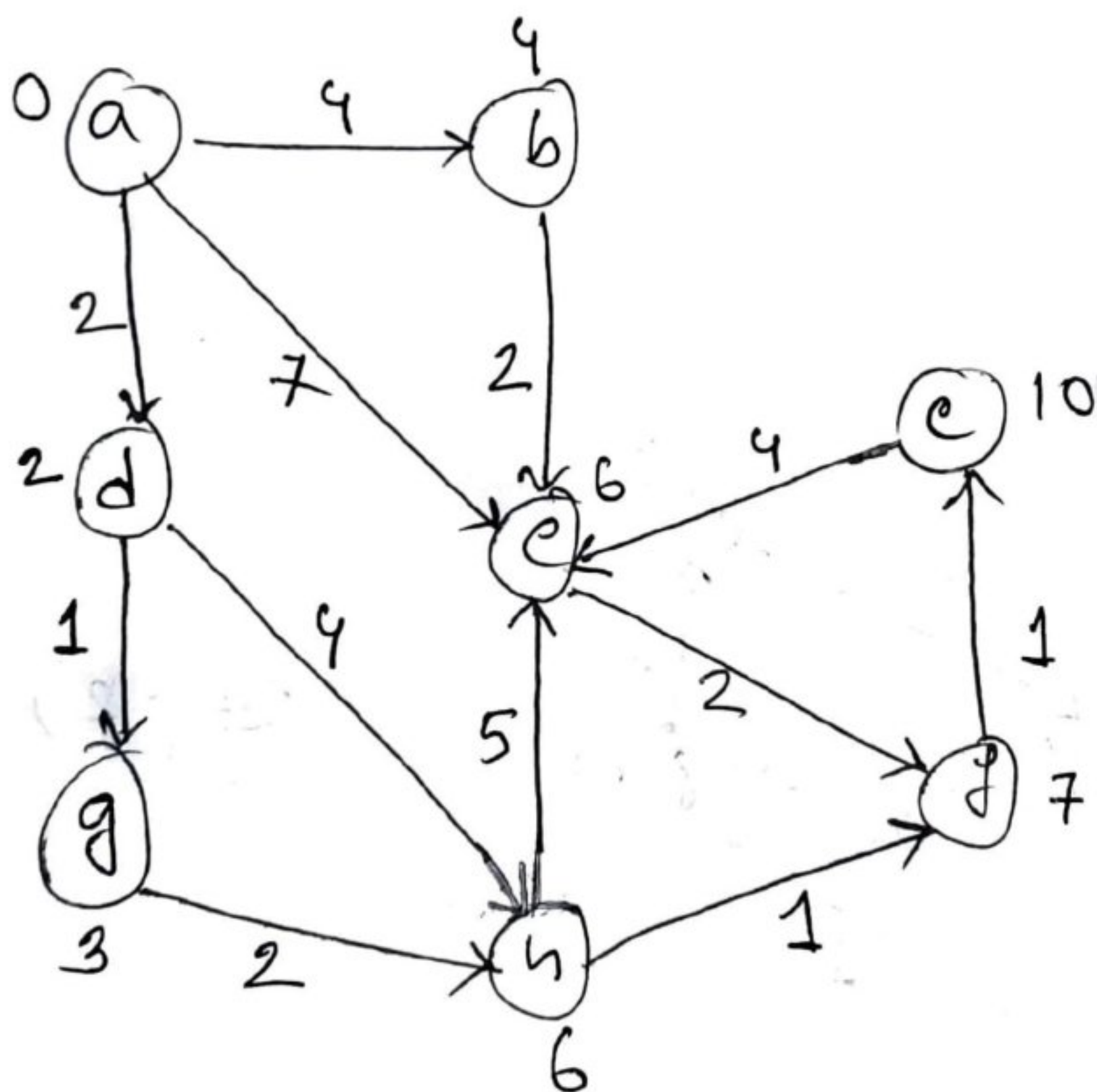
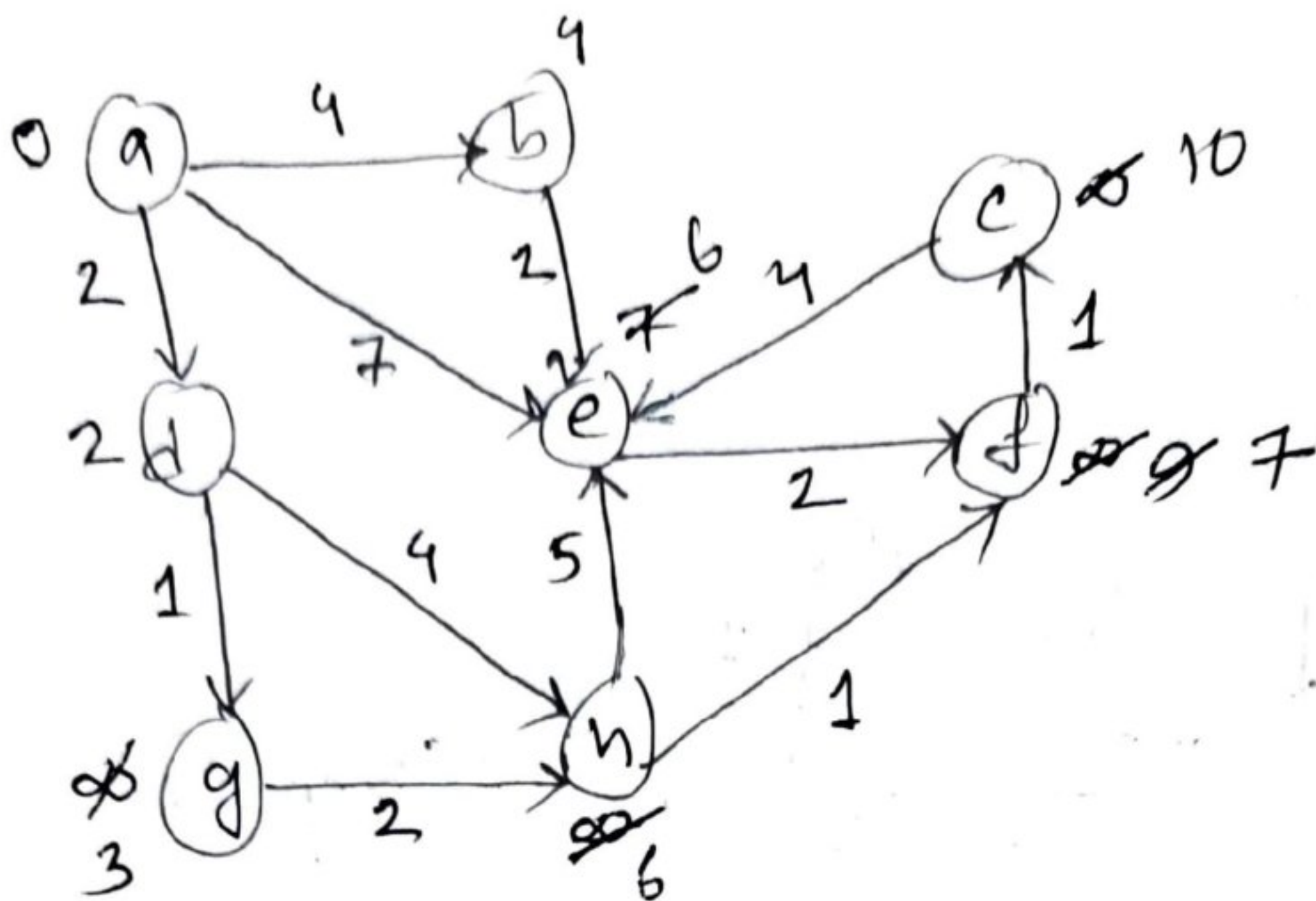


(Step 1)



(Step 2)

(2)



Ars.

(c)

A negative weight cycle in a graph is a closed loop where the sum of the weights of its edges is negative, posing challenges for algorithms that aim to find the shortest paths.

The Bellman-Ford algorithm has a time complexity of $O(V * E)$, where V is the number of vertices and E is the number of edges in the graph. Here's analyzing the running time:

1. Initialization:- This step takes $O(V)$ time to set initial distances.

2. Relaxation:- The algorithm relaxes all edges for a total of $V-1$ iterations. In each iteration, it checks and updates the distance estimates for all edges. Since there are $V-1$ iterations and E edges, the time complexity for relaxation is $O((V-1) * E) = O(V * E)$.

And after $V-1$ iterations, it performs an additional iteration of time complexity $O(E)$ to check for

negative weight cycles.

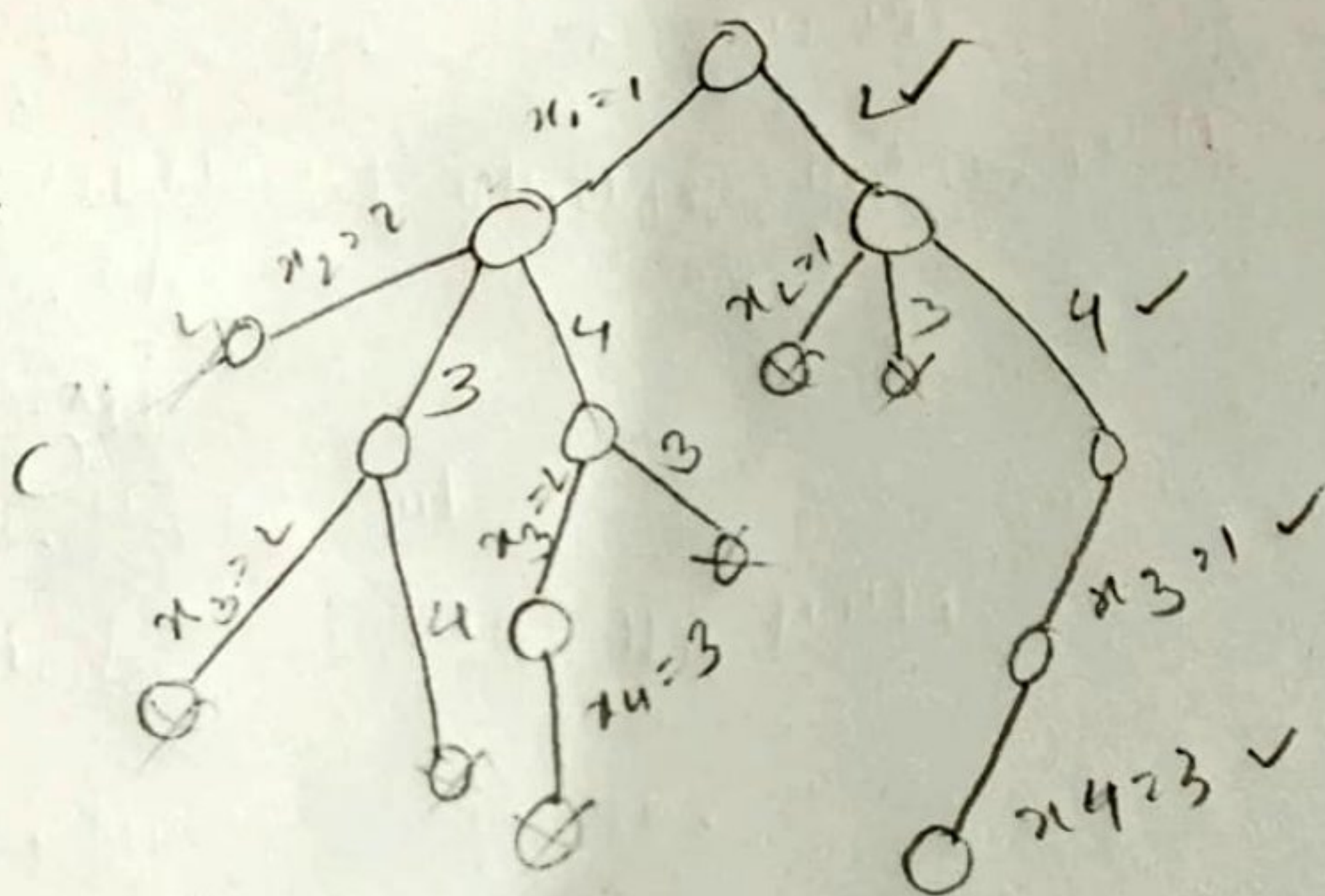
So combining these steps, the over-all time complexity of the Bellman-Ford's algorithm is $O(V) + O(VE) + O(E)$, which simplifies to $O(VE)$.

Thus, the running time of the algorithm is analyzed.

5. (a)

	1	2	3	4
1		Q_1		
2				Q_2
3	Q_3			
4			Q_4	

state space tree



S_1 S_2 S_3 S_4
 ✓

2	4	1	3
---	---	---	---

3	1	4	2
---	---	---	---

5(b) Decision Problem: A decision problem is a type of computational problem that requires a yes/no answer. It involves determining whether a given input satisfies a certain property or constraint. The goal is to decide whether a solution exists for a given instance of the problem. Examples include the traveling salesman problem, where the question is whether there is a tour of a set of cities with a total distance less than a given value.

Optimization Problem: An optimization problem involves finding the best solution among a set of feasible solutions. The goal is to maximize or minimize an objective function, subject to certain constraints. Unlike decision problems, optimization problems seek an optimal solution rather than a simple yes/no answer. The traveling salesman problem can also be an optimization problem where the objective is to find the shortest possible tour.

The statement "Backtracking methodology can be applied to solve any decision problem given sufficient amount of time" is generally correct but needs some qualification-

Versatility: Backtracking is a versatile algorithmic technique that can be applied to a wide range of decision problems. It systematically explores the solution space by making choices at decision points and backtracking when necessary.

Completeness: In theory, backtracking can find a solution for any decision problem, provided there is a solution. It exhaustively searches through all possible combinations of choices.

However, there are considerations:

Time Complexity: The statement qualifies that backtracking can be applied given a sufficient amount of time. The efficiency of backtracking can be impacted by the size of the solution space. For problems with a large number of possibilities, backtracking might become impractical.

Optimization Problems: While backtracking is well-suited for decision problems, it might not be the most efficient approach for large-scale optimization problems. Specialized algorithms like dynamic programming or greedy algorithms may be more suitable.

In summary, while backtracking is a powerful tool for solving decision problems, its practicality depends on the size and complexity of the problem. For very large or complex problems, more efficient algorithms tailored to the specific problem structure might be necessary.

5.C

P Class

The P in the P class stands for **Polynomial Time**. It is the collection of decision problems (problems with a “yes” or “no” answer) that can be solved by a deterministic machine in polynomial time.

NP Class

The NP in NP class stands for **Non-deterministic Polynomial Time**. It is the collection of decision problems that can be solved by a non-deterministic machine in polynomial time.

NP COMPLETE CLASS

A problem is NP-complete if it is both NP and NP-hard. NP-complete problems are the hard problems in NP.

Ans to the Qu no-06 (a)

The four variants of shortest path algorithm

- Bellman-Ford Algorithm
- Dijkstra's Algorithm
- Floyd-Warshall Algorithm
- Topological Sort

In below, these variants are described

1) Bellman-Ford Algorithm: The Bellman-Ford algorithm solves the single-source problem in the general case, where edges can have negative weights and the graph is directed. If the graph is undirected, it will have to be modified by including two edges in each direction to make it directed.

Bellman-Ford has the property that it can detect negative weight cycles reachable from

the source, which would mean that no shortest path exists. If a negative weight cycle existed, a path could run infinitely on that cycle, decreasing the path cost to $-\infty$.

If there is no negative weight cycle then Bellman-Ford returns the weight of the shortest path along with the path itself.

2) Dijkstra's Algorithm: Dijkstra's algorithm makes use of Breadth-first Search (BFS) to solve the single-source problem. It does place one constraint on the graph. There can be no negative weight edges. However, for this one constraint, Dijkstra's greatly improves on the runtime of Bellman-Ford.

Dijkstra's algorithm is also sometimes used to solve the all-pairs shortest path problem by simply running it on all vertices in V .

Again, this requires all edge weights to be positive.

Floyd-Warshall Algorithm: The Floyd-Warshall algorithm solves the all pairs shortest path problem. It uses a dynamic programming approach to do so.

Negative edge weight may be present for Floyd-Warshall.

Floyd-Warshall takes advantage of the following observation:

The shortest path from A to C is either the shortest path from A to B plus the shortest path from B to C or it is the shortest path from A to C that's already been found. This may seem trivial, but it's what allows Floyd-Warshall to build shortest paths from smaller shortest paths, in the classic dynamic programming way.

Topological Sort: For graphs that are directed acyclic graphs (DAGs) a very useful tool emerges for finding shortest paths. By performing a topological sort on the vertices in the graph, the shortest path problem becomes solvable in linear time.

A topological sort is an ordering all of the vertices such that for each edge (u, v) in E , u comes before v in the ordering. In a DAG, shortest paths are always well defined because even if there are negative weight edges, there can be no negative weight cycles. The runtime of these shortest path algorithms given below

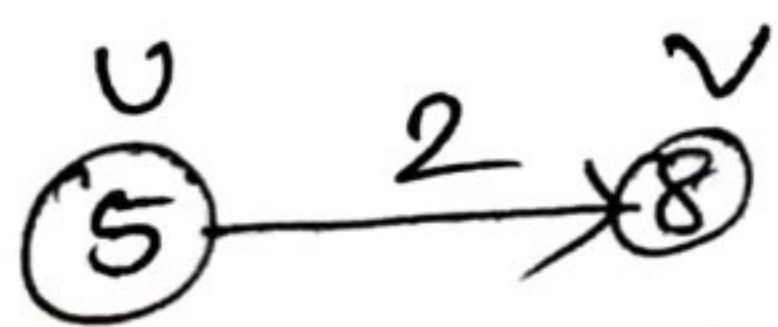
Algorithm	Runtime
Bellman-Ford	$O(V \cdot E)$
Dijkstra	$O(V ^2)$
Topological Sort	$O(V + E)$
Floyd-Warshall	$O(V ^3)$

Ans to the Q no-6 (b)

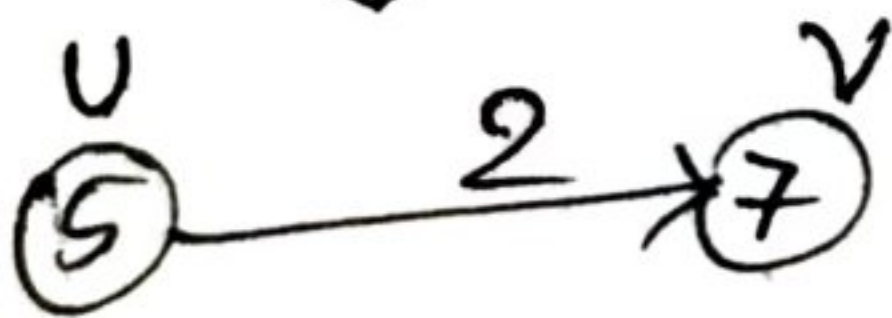
The process of relaxing an edge (u, v) consists of testing whether we can improve the shortest path to v found so far by going through u and if so updating $v.d$ and $v.\pi$. A relaxation step may decrease the value of the shortest path estimate $v.d$ and update v 's predecessor attribute $v.\pi$. The following code performs a relaxation step on edge (u, v) in $O(1)$ time.

RELAX(u, v, w)

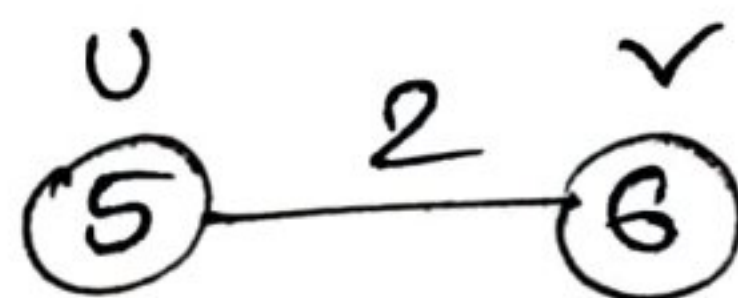
if $v.d > u.d + w(u, v)$
 $v.d = u.d + w(u, v)$
 $v.\pi = u$



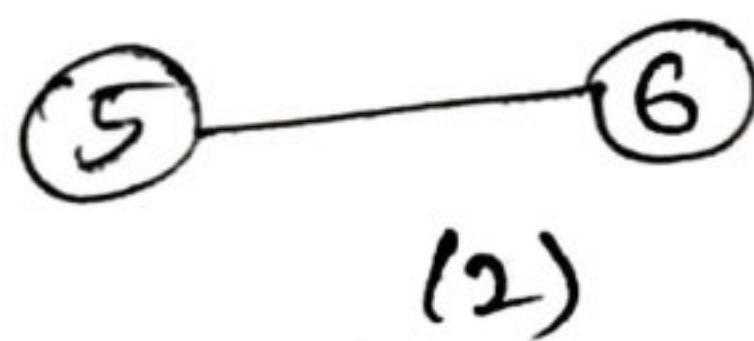
↓ RELAX(u, v, w)



(1)



↓ RELAX(u, v, w)



(2)

Fig: Result after relaxation